

Informe del Proyecto de Compilación

Kelen Alfaro García C311
Adriana Boué García C311
Carlos A. Mazorra Matos C311

june 2026

Introducción

HULK es un lenguaje de programación orientado a objetos con soporte para funciones de primera clase, inferencia de tipos, protocolos estructurales, macros higiénicas y generadores. El compilador traduce programas HULK a código nativo mediante LLVM, pasando por las etapas clásicas de un compilador moderno: análisis léxico, sintáctico y semántico, seguidos de la generación de código intermedio en LLVM IR y su posterior compilación a binario.

El presente informe documenta cada una de estas etapas: la arquitectura del AST, el lexer, el preprocesador, el parser, el módulo semántico, la generación de código y las características adicionales del lenguaje implementadas más allá del núcleo obligatorio. Para cada componente se describen las decisiones de diseño adoptadas, las estructuras de datos empleadas y las relaciones entre módulos.

1 Arquitectura del AST

El AST (*Abstract Syntax Tree*) de HULK es la representación intermedia tipada del programa. Está diseñado para ser:

- Simple de construir desde el parser.
- Estable durante las fases de análisis (minimizar mutaciones directas).
- Suficientemente expresivo para guardar la semántica necesaria por las fases posteriores (tipos, referencias, trazabilidad).

1.1 Estructura Principal

El nodo raíz es `Program`:

- `Program { items: Vec<Item> }`

`Item` recoge tanto declaraciones como expresiones globales:

- `Import(ImportDecl)`
- `Export(ExportDecl)`
- `Function(FunctionDecl)`
- `Type(TypeDecl)`
- `Protocol(ProtocolDecl)`
- `Macro(MacroDecl)`
- `GlobalExpr(Expr)`

Esta separación permite que pases (por ejemplo, análisis de declaraciones, resolución de símbolos o desugaring) trabajen uniformemente sobre la lista de `items` sin ramas especiales.

1.2 Declaraciones Principales

Las declaraciones se representan con structs específicos que contienen la información necesaria para cada caso:

- `ImportDecl` / `ExportDecl`: ruta, alias y metadata.
- `FunctionDecl`: identificador, lista de parámetros (nombre + `TypeRef` opcional), tipo de retorno opcional y cuerpo (`Expr` o `Block`).
- `TypeDecl`: nombre del tipo, parámetros genéricos, lista de miembros (campos y métodos) y herencia/implements.
- `ProtocolDecl`: interfaz que expone firmas; puede heredar otros protocolos.
- `MacroDecl`: nombre, parámetros especiales (`@`, `$`, `*`), y cuerpo que suele ser una expresión o bloque que el macro expandirá.

Estas estructuras modelan el espacio de nombres y sirven como entradas para la fase de resolución de símbolos y la inferencia de tipos.

1.3 Tipos y Anotaciones

Los tipos se representan con `TypeRef`:

- `Number`, `String`, `Boolean` — tipos primitivos.
- `Vector(Box<TypeRef>)` — vectores/postfix `T[]` (soporta anidamiento: `T[] []`).
- `Function(Vec<TypeRef>, Box<TypeRef>)` — tipos de función (`T1, T2 -> R`).
- `Custom(String)` — referencia a tipos definidos por el usuario.

1.4 Expresiones y KindExpr

Las expresiones usan un wrapper uniforme:

```
pub struct Expr {  
    pub id:    NodeId,  
    pub span: Span,  
    pub kind: KindExpr,  
}
```

Separar `kind` de `id`/`span` permite mantener metadatos (identificadores únicos y posición en fuente) sin mezclarlos con la semántica de la expresión.

La fábrica `mk_expr(kind, span)` centraliza la creación de nodos: asigna un `NodeId` fresco, fija el `span` y devuelve la `Expr` completa.

1.4.1 KindExpr — Variantes

`KindExpr` es un `enum` exhaustivo:

- `Literal(LiteralExpr)`
- `Variable(VariableExpr)`
- `Binary(BinaryExpr)`
- `Unary(UnaryExpr)`
- `Call(CallExpr)`
- `BaseCall(BaseCallExpr)`
- `MacroCall(MacroCallExpr)`
- `Let(LetExpr)`
- `Block(BlockExpr)`
- `If(IfExpr)`
- `While(WhileExpr)`
- `For(ForExpr)`
- `Assign(AssignExpr)`
- `MemberAccess(MemberAccessExpr)`
- `Index(IndexExpr)`
- `Array(ArrayExpr)`
- `ArrayComprehension(ArrayComprehensionExpr)`

- `Lambda(LambdaExpr)`
- `New(NewExpr)`
- `Is(IsExpr)`
- `As(AsExpr)`
- `Match(MatchExpr)`

Agregar variantes exige actualizar los pases dependientes (parser, semántica, codegen). Mantener `match` exhaustivos ayuda al compilador a forzar esos cambios mediante compilación fallida en lugar de errores silenciosos.

1.5 Expresiones Específicas

- `BinaryExpr` contiene `left: Expr`, `op: BinaryOp`, `right: Expr`.
- `CallExpr` contiene `callee: Expr` y `args: Vec<Expr>`; si los argumentos usan marcas (`@`, `$`) la acción del parser puede producir `MacroCallExpr`.
- `ArrayExpr` contiene `elements: Vec<Expr>` para literales.
- `ArrayComprehensionExpr` conserva la forma sintáctica original (elemento, variable, iterable, condición opcional) para facilitar trazabilidad y generación diferida.
- `LambdaExpr` guarda parámetros y el cuerpo (expresión o bloque).

Las estructuras recursivas usan `Box` o `Vec` según convenga para evitar tipos de tamaño infinito en Rust.

1.6 Identificadores, Spans y Trazabilidad

- `NodeId` es un `u32` (wrapped) usado como clave estable en mapas de análisis (`HashMap<NodeId, ...>`) para tipos, símbolos, inferencia y anotaciones.
- `Span { start, end }` guarda posiciones en bytes o en offsets de `Location` (según la convención del lexer) y se usa para diagnosticar errores.

2 Arquitectura del Lexer

El módulo de lexing usa `logos`, una biblioteca de lexer orientada a patrones, que convierte el texto fuente en una secuencia de tokens. En HULK esta capa se mantiene deliberadamente simple: su objetivo es reconocer los símbolos léxicos válidos, descartar ruido e informar errores de caracteres no reconocidos. La semántica se resuelve después, en el parser y el preprocesador.

2.1 Definición de Token

El enum `Token` en `src/lexer.rs` define los tokens reconocidos por el lexer. Cada variante puede corresponder a un token fijo, un token con payload o una regla de ignorado.

- Se ignoran espacios en blanco y saltos con `#[logos(skip "[\t\r\n\f]+")]`.
- Se descartan comentarios de línea `//...` y bloques `/* ... */`.
- Literales y símbolos se reconocen y se transforman en tokens con los valores necesarios.

La enumeración incluye categorías clave:

- **Keywords:** `import`, `export`, `let`, `if`, `else`, `while`, `for`, `function`, `type`, `new`, `inherits`, `is`, `as`, `protocol`, `interface`, `extends`, `def`, `define`, `match`, `case`, `default`, `self`, `true`, `false`.
- **Tipos:** `Number`, `String`, `Boolean`.
- **Operadores aritméticos:** `Plus`, `Minus`, `Star`, `Slash`, `Caret`, `Mod`.
- **Operadores de comparación:** `EqualEqual`, `NotEqual`, `LessEqual`, `GreaterEqual`, `Less`, `Greater`.
- **Operadores lógicos y especiales:** `And`, `Or`, `Not`, `Equal`, `ColonEqual`, `At`, `DoubleAt`, `DoublePipe`, `Dollar`.
- **Delimitadores:** `LParen`, `RParen`, `LBrace`, `RBrace`, `LBracket`, `RBracket`, `Comma`, `Dot`, `Colon`, `Semicolon`, `Arrow`, `ThinArrow`.
- **Literales:** `Identifier(String)`, `Number(f64)`, `String(String)`.

En `logos`, el orden de las variantes importa cuando dos patrones pueden solaparse. Por ejemplo, `==` debe reconocerse antes que `=` y `>=` antes que `>`. HULK mantiene esa prioridad en la definición de `Token` para evitar coincidencias incorrectas.

2.2 Reglas Especiales

El lexer aplica reglas concretas a los literales:

- Identificadores siguen `[a-zA-Z][a-zA-Z0-9_]*`.
- Números siguen `[0-9]+(.[0-9]+)?`; es decir, admiten enteros y decimales simples.
- Strings usan delimitadores `"..."` e interpretan secuencias escapadas limitadas: `\n`, `\t`, `\r`, `\\` y `\"`.

Además:

- Los literales numéricos se parsean directamente a `f64` para que el parser reciba valores ya convertidos.
- Las cadenas se desescapan en el lexer y se guardan como `String` limpio.
- Los comentarios no generan tokens; se eliminan antes de que el parser procese la entrada.

3 Preprocesador y Normalizaciones

El módulo `src/preprocessor.rs` es clave para la robustez del front-end. Su responsabilidad es aplicar validaciones y transformaciones que simplifican el parser.

3.1 Validaciones Específicas

`validate_array_initializer_syntax` es una comprobación temprana cuyo objetivo no es solamente rechazar código mal formado, sino asegurar que la sintaxis de inicializadores de array llega al parser en la forma esperada.

En HULK la forma original del inicializador de arrays usa `{ ... }` para distinguirse de llamadas y otros bloques. Antes de que el parser procese el código, el preprocesador puede convertir esa sintaxis original a una forma interna menos ambigua. Por eso el validador hace dos cosas:

- Rechaza directamente expresiones que ya usan `()` o `[]` en lugar de la forma original `{ }` en contextos de inicializador, porque ese código no debería escribirse así y puede producir ambigüedad.
- Señala el error antes de que el lexer/parser intente procesarlo, para evitar diagnósticos confusos derivados de reglas sintácticas equivocadas.

Casos concretos:

- **Caso 1:** `new Type[5] (i -> i + 1)` — esta forma parece un inicializador lambda, pero la sintaxis original válida es `new Type[5] {i -> i + 1}`. El validador rechaza la forma con paréntesis y obliga al usuario a escribir con llaves. Si el código usa `{ ... }`, el preprocesador lo normaliza a la forma interna que el parser acepta.
- **Caso 2:** `let a: Number[] = [10, 20, 30]` — el literal de array válido en el dialecto original se escribe con llaves `{10, 20, 30}`. El validador marca este uso como inválido y sugiere la forma correcta.

3.2 Normalización de Sintaxis Azucarada

El preprocesador implementa transformaciones sintácticas (*azúcar*) de forma explícita y local para que el parser trabaje sobre una sintaxis más regular y previsible. Estas transformaciones se realizan en una o dos pasadas sobre el texto fuente.

Transformaciones principales:

- `new Type[expr]{ ... } → new Type[expr]( ... )`: cuando se detecta el patrón de un inicializador lambda escrito con llaves, se convierte a una forma con paréntesis y una función lambda explícita. Esta normalización evita que el parser tenga que soportar una producción adicional y facilita la desambiguación entre llamadas e inicializadores.
- `{a, b, c} → [a, b, c]` para literales de array: en los casos donde el contenido del bloque corresponde claramente a una lista de expresiones separadas por comas, el preprocesador convierte llaves en corchetes para unificar la representación de arrays literales en el parser.

Las normalizaciones solo se aplican cuando el preprocesador puede decidir de forma local y segura que no está transformando un bloque de código válido. Las transformaciones quedan registradas para facilitar depuración y pruebas automáticas.

3.3 Decisión de Diseño del Preprocesador

La decisión de extraer las transformaciones sintácticas a un módulo independiente responde a motivos prácticos y de mantenibilidad:

- **Simplicidad del lexer**: permite que el lexer sea una máquina de estados orientada a reconocer patrones lexemáticos y no una capa que intente resolver ambigüedades sintácticas.
- **Reducción de la gramática**: al normalizar azúcar y casos especiales antes del parsing, `parser.lalrpop` puede permanecer conciso y se evitan producciones excepcionales que complican la tabla de parseo.
- **Mejora en la calidad de los diagnósticos**: el preprocesador puede detectar patrones mal formados y proporcionar mensajes de error más claros y específicos que si esos problemas se detectaran como errores sintácticos genéricos dentro del parser.

En iteraciones previas se observaron múltiples fallos donde pequeñas variaciones sintácticas inducían conflictos en `lalrpop` (shift/reduce o reduce/reduce). Para mitigarlos sin sacrificar expresividad, se decidió mover la responsabilidad de desambiguación a una capa previa, donde las transformaciones son explícitas y fáciles de testear. El flujo resultante queda más modular:

lectura → preprocesador → lexer → postprocesador de tokens → parser

4 Arquitectura del Parser

`src/parser.lalrpop` define la gramática del lenguaje y mapea sus producciones directamente a nodos del AST. Esta capa transforma la secuencia de tokens en un árbol rico en estructura, manteniendo información de ubicación y de origen para diagnóstico posterior.

4.1 Integración con Tokens y AST

El parser declara la interfaz de tokens para `lalrpop` con:

```
extern {
    type Location = usize;
    type Error    = ();
    enum Token { ... }
}
```

Esto le dice a `lalrpop` cómo leer tokens del lexer y detectar errores de análisis. Cada producción devuelve un `Expr`, `Item`, `TypeRef` u otro nodo del AST, y usa `mk_expr(...)` para asignar `NodeId`/`Span` automáticamente.

El parser no crea estructuras auxiliares complejas: cada expresión se construye inmediatamente como un nodo AST. Por ejemplo, una regla de suma se compila en `mk_expr(KindExpr::Binary(BinaryExpr{left, op, right}), span)`, enlazando subexpresiones y preservando la posición en el código fuente.

4.2 Programa y Elementos Principales

El nodo raíz `Program` puede representarse en distintas formas según el contenido del archivo:

- `TopLevelItems` seguido de `GlobalExprItems`.
- Solo `TopLevelItems`.
- Una sola `Expr` cuando el archivo contiene únicamente una expresión.

Esto permite escribir programas con declaraciones primero y luego código ejecutable global, o bien scripts cortos que son solo una expresión.

`TopLevelItem` agrupa las declaraciones válidas en la parte superior del archivo: `ImportDecl`, `ExportDecl`, `MacroDecl`, `ProtocolDecl`, `TypeDecl` y `FunctionDecl`. Al mismo tiempo, el símbolo `Item` admite `Expr` ";" como elemento del programa, habilitando archivos en que se mezclan definiciones y ejecución inmediata:

```
import foo
let x = 5;
print(x);
```


4.3 Declaraciones Detalladas

La gramática soporta declaraciones complejas:

- **MacroDecl** con **def** o **define**, permitiendo parámetros especiales *****, **@**, **\$** y cuerpo en expresión o bloque.
- **TypeDecl** con parámetros genéricos, herencia **inherits** y miembros que pueden ser campos o métodos.
- **ProtocolDecl/Interface** con herencia de protocolos y firmas de método.
- **FunctionDecl** con cuerpo que puede ser una sola expresión o bloque de múltiples expresiones.

Este conjunto de opciones hace posible un lenguaje con macros de primera clase, tipos estructurados y extensibilidad por herencia/protocolos.

4.4 Expresiones y Precedencia

El parser organiza las expresiones en niveles de precedencia para que las operaciones se interpreten con el orden esperado:

1. **LogOrExpr** y **LogAndExpr** para **|** y **&**.
2. **EqExpr** para **==**, **!=**.
3. **CmpExpr** para **<**, **>**, **<=**, **>=**.
4. **TypeExpr** para **is** y **as**.
5. **StringExpr** para concatenación con **@** y **@@**.
6. **ArithExpr** para suma y resta.
7. **MulExpr** para multiplicación, división y módulo.
8. **PowExpr** para exponentes.
9. **UnaryExpr** para negación y **not**.
10. **CallExpr** para llamadas, acceso a miembros e indexación.
11. **PrimaryExpr** para literales, variables, arrays, paréntesis y comprensiones.

Esta jerarquía evita ambigüedades comunes en expresiones mixtas. Por ejemplo, en la expresión `1 + x * y - z`, el operador `*` tiene mayor precedencia que `+` y `-`: el parser construye un **MulExpr** con `x` e `y` primero, luego lo combina con `1`, y finalmente resta `z`.

4.5 Constructores Especiales

El parser incluye reglas dedicadas para construcciones de control y formas sintácticas especiales:

- `IfExpr`, `WhileExpr`, `ForExpr`, `LetExpr`, `BlockExpr`.
- `LambdaExpr` con la forma `function(...) -> expr`.
- `NewExpr` con varias variantes:
 - `new BaseType(args)` para instancias normales.
 - `new BaseType[size] (params -> body)` para inicializadores lambda de arrays.
 - `new BaseType[size]` y `new BaseType[]` para arrays sin inicializador.
- `MatchExpr` para coincidencia de patrones, inclusivo de literales y operadores.

4.6 Manejo de Macros y Bloques

El parser distingue llamadas normales de macros basándose en argumentos marcados:

- `MacroMarkedArgList` acepta argumentos prefijados con `@` (argumento simbólico) o `$` (placeholder).
- Si el callee es un identificador y la lista de argumentos contiene marcas, se construye un `MacroCallExpr`.
- En caso contrario, se construye un `CallExpr` normal.

Esto hace posible que sintaxis como `repeat(@x)` o `print($msg)` se parseen como macros, mientras que `foo(x, y)` sigue siendo una llamada normal.

4.7 Tipos y Arreglos

La gramática de tipos admite:

- Tipos básicos y personalizados: `Number`, `String`, `Boolean`, `Custom(name)`.
- Vectores con sufijo `[]`, que pueden repetirse para representar `T[][]`.
- Tipos de función con forma `(T1, T2) -> R`.

Un tipo como `Number[][]` se modela como `Vector(Vector(Number))`, lo que permite representar anidamiento de arrays de forma natural.

4.8 Literales, Variables y Arrays

`PrimaryExpr` cubre los casos primarios más simples: literales numéricas, de cadena y booleanas; `self` y variables identificadas; expresiones entre paréntesis; y literales de array y comprensiones.

Las comprensiones de arrays admiten la sintaxis:

```
[ element | variable in iterable ]
```

El parser crea un nodo `ArrayComprehensionExpr` que conserva el elemento, la variable, el iterable y cualquier condición. Esta forma de alto nivel se mantiene hasta que la fase de desugaring o codegen decide cómo bajar la comprensión a código ejecutable.

4.9 Diseño y Extensibilidad

El parser está construido para ser modular y fácil de extender:

- Cambios en la gramática se reflejan directamente en las reglas de `parser.lalrpop`.
- Las acciones se mantienen pequeñas y producen nodos AST simples.
- Las nuevas construcciones se integran añadiendo variantes de AST y reglas de parseo, con un impacto mínimo en el resto del pipeline.

El resultado es una cadena de parseo que mantiene la sintaxis del lenguaje clara y el AST expresivo, al tiempo que deja la mayor parte de la semántica compleja para fases posteriores.

5 Módulo Semántico

El módulo semántico es el encargado de validar un programa HULK después de su fase de parseo. Sus responsabilidades principales incluyen:

- Resolución de nombres y ámbitos (*scopes*).
- Inferencia de tipos y verificación estática de consistencia.
- Validación de reglas de control de flujo y asignación definitiva de variables.
- Vinculación semántica de módulos mediante declaraciones `import` y `export`, gestionando la carga de dependencias, resolución de espacios de nombres y filtrado de la API pública.
- Preparación del contexto de información estructurada requerida por las fases posteriores del compilador (como la generación de código).

Se encuentra implementado bajo la ruta `hulk/src/semantic/` y está dividido en submódulos especializados para maximizar su mantenibilidad.

5.1 Componentes del Módulo Semántico

5.1.1 Orquestación Central e Inter-Módulos (`src/semantic/mod.rs`)

Gestiona el estado global del análisis, el orden de las pasadas de recolección y la vinculación de archivos externos.

- **SemanticAnalyzer**: estructura central que posee las tablas de símbolos, el recolector de diagnósticos, los mapas de tipos inferidos, los estados de asignación de variables y las formas de tipos y protocolos.
- **Sistema de Módulos (import/export)**: administra la carga diferida o ansiosa (*eager*) de dependencias mediante `module_cache`. Utiliza `namespaces` para aislar los símbolos de cada módulo y `loading_modules` para detectar de forma estricta los ciclos de importación circular.
- **Filtrado de API Pública**: al finalizar la carga de un módulo, la función `build_public_namespace` expone únicamente los símbolos declarados explícitamente en bloques `export`. Si no se especifican exportaciones, mantiene la compatibilidad publicando todos los símbolos de primer nivel.
- **SemanticContext**: estructura inmutable de salida que consolida los tipos inferidos (`inferred_types`), las formas de las clases (`type_shapes`), las formas de los protocolos (`protocol_shapes`), el mapa global de símbolos (`global_symbols`) y las listas de importaciones y exportaciones.

5.1.2 Análisis de Expresiones y Patrones (`src/semantic/expr.rs`)

Se encarga de recorrer recursivamente el AST de las expresiones para validar su semántica y determinar sus tipos.

- **Validación de Funciones y Métodos**: controla que los cuerpos de las funciones se correspondan con los tipos declarados e infiere sus valores de retorno.
- **Emparejamiento de Patrones (Match)**: analiza las expresiones `Match` y sus ramas evaluando la validez de las estructuras `Pattern::Identifier`.
- **Estrechamiento de Tipos (*Type Narrowing*)**: implementa la función `extract_is_narrowing`, la cual deduce si una validación de tipo en tiempo de ejecución (como el operador `is`) permite restringir el tipo estático de una variable dentro de un bloque condicional o rama de un `match`.

5.1.3 Control de Flujo y Evaluación Constante (`src/semantic/flow.rs`)

Implementa las reglas de análisis de flujo de ejecución y evaluación en tiempo de compilación.

- **Garantías de Retorno y Valor**: utiliza la función `guarantees_value` para asegurar que todas las ramas de una expresión (como un `if` o `match`) produzcan un valor válido antes de continuar la ejecución.

- **Detección de Ciclos Infinitos:** mediante `definitely_non_terminating`, el compilador detecta casos comprobables estáticamente de no terminación (por ejemplo, bucles con condiciones siempre verdaderas).
- **Evaluación Constante:** contiene la lógica (`eval_const_number`, `eval_const_bool`, `eval_const_literal`) para pre-evaluar expresiones deterministas en tiempo de compilación, permitiendo optimizar el análisis y emitir advertencias sobre ramas inalcanzables (`report_unreachable_if_branches`).

5.1.4 Desazucarización de Funtores (`src/semantic/functor_desugar.rs`)

Este módulo reescribe y simplifica las expresiones de alto nivel relacionadas con funciones anónimas, clausuras y bloques ejecutables complejos. Transforma estas estructuras en clases y métodos tradicionales equivalentes para simplificar el trabajo de las fases de generación de código.

5.1.5 Inferencia de Tipos Basada en Restricciones (`src/semantic/inference.rs`)

Implementa un sistema de inferencia basado en restricciones inspirado en Hindley-Milner, adaptado a la naturaleza orientada a objetos y estructural de HULK.

- **SymbolRequirements:** estructura que acumula las restricciones implícitas descubiertas durante el uso de un símbolo indeterminado. Registra si el símbolo posee un tipo concreto conocido (`concrete`), qué métodos ha invocado y con qué aridad (`methods`), si requiere comportarse como una función (`requires_function`), o si sus operaciones exigen que sea un contenedor indexable (`requires_vector`).
- **Síntesis Dinámica de Protocolos:** cuando un símbolo no anotado invoca métodos de forma estructural, el inferidor genera un protocolo sintético mediante `next_synthetic_protocol_name`. Estos nombres internos siguen el patrón `_P{n}` (por ejemplo, `_P1`, `_P2`) para encapsular las restricciones contractuales del símbolo.

5.1.6 Expansor Higiénico de Macros (`src/semantic/macro_expander.rs`)

Gestiona la fase de transformación sintáctica del AST antes de la tipificación.

- **Validación de Parámetros:** comprueba que el número de argumentos coincida exactamente con la definición de la macro y gestiona los parámetros especiales como bloques traseros (*trailing blocks*) validados mediante `MacroParamKind::Block`.
- **Higiene de Macros:** implementa un algoritmo de renombrado en `expand_pattern` que genera identificadores frescos (`fresh_name`) para todas las variables locales introducidas por la macro, previniendo la captura accidental de nombres en el sitio de la llamada.

- **Detección de Recursión:** utiliza una pila de expansión (`expansion_stack`) para rastrear invocaciones anidadas y emitir errores semánticos si se detecta una macro recursiva infinita.

5.1.7 Gestión de Ámbitos y Tabla de Símbolos

(`src/semantic/scope.rs` y `src/semantic/symbol_table.rs`)

Proporcionan la infraestructura para el manejo de entornos léxicos, el estado de las variables y su asignación definitiva.

- **SymbolTable:** almacena las asociaciones activas entre identificadores y sus metadatos (`SymbolKind` y `SemanticType`).
- **Control de Sombreado (*Shadowing*):** las funciones de inserción local verifican si el nuevo símbolo oculta una declaración previa de un ámbito externo, emitiendo una advertencia si es el caso.
- **Estados de Asignación y Solo Lectura:** realiza un seguimiento riguroso (`assigned_scopes`) para garantizar que ninguna variable sea leída antes de ser inicializada. Emplea `define_local_with_state` para registrar si una variable nace asignada, consolida los estados de flujos condicionales con `merge_definite_assignment_states`, y protege símbolos que no pueden ser reasignados, como `self`, mediante `readonly_scopes`.

5.1.8 Verificación de Tipos y Operaciones de Vectores

(`src/semantic/type_checks.rs`)

Contiene las reglas de consistencia de tipos y resolución de miembros.

- **Conformidad de Tipos:** proporciona utilidades como `is_compatible_type` y `expect_type` para asegurar que las asignaciones, retornos y evaluaciones respeten los contratos del sistema nominal y estructural.
- **Soporte Nativo de Vectores:** expone `resolve_vector_member_type`, que inyecta y valida automáticamente los métodos incorporados de las colecciones indexadas: `size()` (retorna `Number`), `next()` (retorna `Boolean`) y `current()` (retorna el tipo base encapsulado por el vector).
- **Herencia de Protocolos:** implementa `protocol_extends`, función que determina si un protocolo hereda de otro navegando la jerarquía declarada mediante un conjunto de visitados (`HashSet`) para evitar bucles infinitos ante definiciones cíclicas.

5.1.9 Definiciones del Sistema de Tipos (`src/semantic/types.rs`)

Define la enumeración `SemanticType`, que modela las entidades del sistema de tipos de HULK:

- **Tipos Atómicos:** `Number`, `String`, `Boolean` y el ancestro común `Object`.

- **Tipos Complejos:** `Function(Vec<SemanticType>, Box<SemanticType>)` para firmas de llamadas, `Vector(Box<SemanticType>)` para arreglos y `Custom(String)` para clases o protocolos definidos por el usuario.
- **Estados Especiales:** `Unknown`, usado como marcador de posición durante fases intermedias de inferencia.

5.2 Decisiones de Diseño: Pattern Matching vs. Visitor

La elección de *pattern matching* sobre el patrón *Visitor* tradicional (común en compiladores escritos en lenguajes orientados a objetos como Java o C#) responde a principios arquitectónicos y al aprovechamiento de las garantías estáticas del lenguaje de implementación:

1. **Idoneidad Idiomática y Modelado de Datos:** los nodos del AST se modelan de forma natural mediante tipos de datos algebraicos (`enum` en Rust) en lugar de una jerarquía de clases polimórficas. El *pattern matching* es una característica nativa de primera clase, eliminando la necesidad de simular doble despacho (*double dispatch*) mediante `expr.accept(visitor)`.
2. **Exhaustividad en Tiempo de Compilación:** el compilador garantiza que todas las variantes de un `enum` sean cubiertas en cada expresión `match`. Si en el futuro se introduce un nuevo nodo sintáctico, el compilador señalará cada punto del análisis semántico que requiere actualización, mitigando errores en tiempo de ejecución.
3. **Localidad y Cohesión de la Lógica:** el *Visitor* facilita agregar nuevas operaciones sobre una jerarquía estable de nodos, mientras que el *pattern matching* facilita evolucionar los nodos y garantiza exhaustividad. En HULK, donde el AST está definido como enums de Rust, el *pattern matching* reduce código repetitivo y mantiene las reglas semánticas cerca de la representación algebraica.
4. **Desestructuración Profunda y Guardas Semánticas:** el análisis de HULK exige lidiar con construcciones complejas como el estrechamiento de tipos y la concordancia de patrones de identificadores. `match` permite realizar desestructuraciones anidadas y aplicar cláusulas de guarda condicionales (`if`) en una sola expresión, evitando el código repetitivo que requeriría una jerarquía de visitantes.

5.3 Estrategia de Manejo de Errores

La fase semántica está diseñada bajo una filosofía de **acumulación de diagnósticos** en lugar de interrupción temprana (*fail-fast*):

- Los errores locales detectados en expresiones o firmas se registran inmediatamente en el `DiagnosticCollector`.

- El analizador intenta recuperarse del error (por ejemplo, asignando provisionalmente el tipo `SemanticType::Unknown`) y continúa procesando el resto del AST.
- Al completarse todas las pasadas de validación, si el recolector contiene al menos un error, el resultado final de `analyze_program` se consolida como `Err(Vec<Diagnostic>)`.

Esta aproximación maximiza la productividad del desarrollador al reportar múltiples problemas semánticos en una única corrida de compilación.

5.4 Limitaciones Actuales

A pesar de la robustez del análisis semántico, la implementación actual presenta ciertas limitaciones:

1. **Inmutabilidad Basada en Estado Externo:** el sistema de control para variables de solo lectura (`readonly_scopes`) no está integrado directamente en los metadatos principales de la tabla de símbolos, lo que requiere estructuras auxiliares de rastreo e incrementa la complejidad de coordinación.
2. **Separación entre Tipos Explícitos e Inferidos:** los protocolos generados dinámicamente (`_P{n}`) comparten espacio con las declaraciones de tipos tradicionales dentro del contexto semántico. Una separación más explícita permitiría diferenciar contratos generados por inferencia de declaraciones realizadas por el usuario.
3. **Ausencia de Identificadores Semánticos Estables:** los identificadores únicos (`NodeId`) están orientados principalmente a expresiones. La falta de IDs estables para declaraciones de alto nivel limita futuras extensiones como compilación incremental o herramientas externas de análisis.
4. **Identidad de Módulos Dependiente de Rutas:** la caché de módulos depende de la representación de las rutas de archivos. Sin canonicalización, pueden aparecer duplicaciones de carga en proyectos con múltiples raíces o enlaces simbólicos.
5. **Propagación de Unknown:** el uso de `SemanticType::Unknown` permite continuar el análisis tras errores semánticos, pero puede ocultar errores secundarios o producir diagnósticos menos precisos cuando una restricción depende de información perdida.

5.5 Futuras Extensiones Sugeridas

1. **Calificadores de Inmutabilidad Explícitos:** integrar banderas de mutabilidad directamente en los metadatos de los símbolos, eliminando la necesidad de mapas de estado paralelos.

2. **Exposición de Metadatos de Síntesis:** separar formalmente los protocolos sintéticos (`_P{n}`) en un campo dedicado dentro de `SemanticContext` para facilitar optimizaciones en el backend.
3. **Identificadores Semánticos Estables:** implementar IDs únicos no solo para expresiones sino también para declaraciones de primer nivel, facilitando la compilación incremental y el soporte de herramientas de lenguaje externas.
4. **Normalización Canónica de Rutas:** implementar la resolución y canonicalización absoluta de rutas para las claves de `module_cache`, evitando doble carga en configuraciones con raíces de módulos personalizadas o enlaces simbólicos.

6 Generación de Código

6.1 Arquitectura General

La generación de código transforma el AST desazucarado y anotado semánticamente en LLVM IR mediante la biblioteca `inkwell`, un *wrapper* de Rust sobre la API de LLVM. El componente central es la estructura `CodeGenerator<'ctx>`, parametrizada por el lifetime del contexto LLVM, que centraliza todo el estado necesario durante la emisión de IR:

- `context / module / builder`: el contexto LLVM (propietario de tipos e instrucciones), la unidad de compilación (todo el IR se emite en un único módulo) y el *builder* que mantiene el *insertion point* actual. El builder se reposiciona explícitamente al entrar en cada bloque básico.
- `scopes`: pila de tablas de símbolos locales. Cada entrada es un `HashMap<String, VarInfo>` que asocia un nombre de variable a su slot `alloca` y su `ValueKind`.
- `functions`: firmas de todas las funciones y métodos ya declarados en el módulo, usada para emitir llamadas hacia adelante y para construir tipos de función en dispatch por vtable.
- `type_decls, struct_types, vtable_types, vtable_globals`: datos de los tipos HULK definidos por el usuario, sus structs LLVM y sus vtables como variables globales.
- `type_ids, method_orders`: identificadores enteros únicos por tipo y el orden canónico de métodos en la vtable de cada tipo, precalculados antes de emitir código.
- `current_type, current_method`: contexto de la función en compilación, necesario para resolver referencias a `self` dentro de métodos.

El pipeline de generación se ejecuta en el siguiente orden:

1. Recolección de declaraciones de tipos.
2. Creación de structs LLVM y vtables (con asignación de *type_ids* y órdenes canónicos de métodos).
3. Declaración de todas las firmas de funciones y métodos (antes de definir cuerpos, para habilitar recursión mutua y referencias hacia adelante).
4. Definición de los cuerpos.
5. Generación de la función `main`.

6.2 Relación con el Análisis Semántico

El generador recibe dos entradas: el `Program` (AST desazucarado) y un `SemanticAnalysis`. Este segundo objeto contiene toda la información de tipos inferida por la fase semántica; el codegen la consume sin reinferir nada:

- `inferred_types`: `HashMap<NodeId, SemanticType>`: cada nodo del AST tiene un `NodeId` único. Esta tabla mapea cada nodo al tipo que el analizador semántico le asignó. El codegen consulta `analysis.inferred_types.get(&expr.id)` para conocer el tipo de cualquier subexpresión.
- `global_symbols`: firmas de funciones globales, usadas para construir los tipos LLVM al declararlas.
- `type_shapes`: métodos y sus tipos por tipo concreto, usados para construir vtables y resolver herencia.
- `protocol_shapes`: métodos declarados en cada protocolo, usados para identificar tipos conformes en tiempo de compilación.

Esta separación es importante: el codegen **no realiza ninguna inferencia de tipos propia**. Toda decisión semántica ya está resuelta; el codegen solo consulta los resultados. Esto simplifica la generación de código y garantiza que los errores de tipos sean detectados antes de intentar emitir IR.

6.3 Generación Recursiva del AST

La función central del codegen es:

```
lower_expr(&Expr, &SemanticAnalysis) -> Result<CodegenValue, String>
```

Implementa una recursión estructural sobre el AST: cada variante de `KindExpr` produce la secuencia de instrucciones LLVM correspondiente. El IR se construye de abajo hacia arriba (*bottom-up*); cada llamada recursiva retorna un `CodegenValue` que el nodo padre usa como operando. Para ilustrar cómo esta recursión construye el IR, considérese la siguiente función HULK:

```
function area(r: Number): Number => let pi := 3.14159 in pi * r * r;
```

El árbol de llamadas a `lower_expr` y el IR emitido en cada nivel es:

```
lower_expr(Let "pi" = 3.14159 in BinOp(BinOp(pi, *, r), *, r))
|
| ; alloca para pi en el entry block:
| %pi = alloca double
|
+- lower_expr(3.14159)
|   ret: CodegenValue::Number(double 3.14159)
|   store double 3.14159, double* %pi
|
+- lower_expr(BinOp(BinOp(pi, *, r), *, r))
|
| +- lower_expr(BinOp(pi, *, r))
| |
| | +- lower_expr(pi)  => %pi_0 = load double, double* %pi
| | +- lower_expr(r)   => %r_0  = load double, double* %r
| |   %t0 = fmul double %pi_0, %r_0
| |
| +- lower_expr(r)      => %r_1  = load double, double* %r
|   %t1 = fmul double %t0, %r_1
|   ret double %t1
```

El IR resultante para el cuerpo de la función es:

```
define double @area(double %r_arg) {
entry:
  %r    = alloca double
  %pi   = alloca double
  store double %r_arg,    double* %r
  store double 3.14159,   double* %pi
  %pi_0 = load double, double* %pi
  %r_0  = load double, double* %r
  %t0   = fmul double %pi_0, %r_0
  %r_1  = load double, double* %r
  %t1   = fmul double %t0, %r_1
  ret double %t1
}
```

Nótese que `r` se carga dos veces (`%r_0` y `%r_1`): la generación recursiva emite una carga por cada uso de la variable, sin intentar reutilizar valores. El paso `mem2reg` de LLVM elimina estas cargas redundantes promoviendo los `allocas` a registros SSA y propagando los valores, produciendo IR equivalente al que se habría escrito manualmente.

6.4 Representación de Valores

Se definieron dos enums complementarios. `ValueKind` clasifica los tipos en tiempo de compilación del compilador, sin contener valores LLVM:

ValueKind	Tipo LLVM	Descripción
Number	double	Número de punto flotante
Bool	i1	Booleano
String	i8*	Puntero a cadena C
Object	i8*	Puntero opaco a objeto
Vector	%VectorStruct*	Puntero a struct de vector
Closure	%ClosureStruct*	Puntero a struct de clausura

`CodegenValue` envuelve el valor LLVM concreto junto con su variante. Que `String` y `Object` compartan `i8*` es intencional: todos los objetos son punteros opacos en ABI, pero la distinción semántica persiste en `ValueKind` para emitir instrucciones de carga/almacenamiento correctas y para que las coerciones implícitas no requieran conversión de representación.

6.5 Variables, Scoping y el uso de `alloca`

6.5.1 Por qué `alloca` en lugar de SSA puro

LLVM trabaja en forma SSA (*Static Single Assignment*): cada registro virtual se asigna exactamente una vez. El problema con las variables mutables de HULK (operador `:=`) es que, en SSA puro, cada reasignación requeriría un registro distinto, y en puntos de convergencia de flujo de control habría que insertar nodos ϕ explícitos, lo que requiere calcular información de dominancia en el compilador.

La solución estándar en compiladores basados en LLVM es usar `alloca`: reservar un slot de pila para cada variable y operar mediante `store/load`. Esto convierte el problema de SSA en uno de memoria, que LLVM resuelve automáticamente mediante el paso de optimización `mem2reg` (*memory-to-register*), que promueve las `allocas` a registros SSA e inserta los nodos ϕ donde son necesarios. Clang, rustc y la mayoría de compiladores modernos siguen esta misma estrategia.

6.5.2 Implementación

Todas las `allocas` se emiten en el bloque `entry` de la función, no en el punto de uso, requisito de `mem2reg` para que el análisis de dominancia las identifique correctamente. Al entrar a un bloque anidado se hace `push` de un scope; al salir, `pop`. La resolución de nombres busca de interno a externo en la pila de scopes, implementando el alcance léxico de HULK. Las asignaciones `:=` emiten un `store` al slot existente.

6.6 Expresiones Básicas

Aritmética y comparación. Las operaciones numéricas se mapean directamente a instrucciones de punto flotante: `fadd`, `fsub`, `fmul`, `fdiv`. La exponenciación (^) usa la intrínseca `llvm.pow.f64`. Las comparaciones emiten `fcmp` con predicados IEEE (`oeq`, `olt`, `ogt`, etc.).

Operaciones booleanas. `&&` y `||` se emiten con evaluación en cortocircuito: se genera un bloque condicional que salta al resultado sin evaluar el segundo operando si el primero ya lo determina.

Cadenas. El operador de concatenación `@@` invoca la función de runtime `hulk_concat`, que retorna un nuevo `i8*`. Las funciones matemáticas intrínsecas (`sqrt`, `sin`, `cos`, `exp`, `log`) se mapean a intrínsecas LLVM (`llvm.sqrt.f64`, etc.).

6.7 Vectores y Arrays

6.7.1 Literales de vector

La sintaxis `[e1, e2, e3]` produce un nodo `ArrayExpr` en el AST. El codegen aloca un `VectorStruct` en el heap (mediante `malloc`) con tres campos: longitud (`i64`), puntero al buffer de datos (`i8*`) y cursor de iteración (`i64`). El buffer se aloca por separado con tamaño `elem_size * n`, y cada elemento se evalúa recursivamente y se almacena en la posición correspondiente.

6.7.2 Creación con tamaño dinámico

La expresión `new T[size]` no tiene elementos conocidos en tiempo de compilación. El codegen evalúa `size` (un `double` que se convierte a `i64`) y aloca el buffer con `elem_size * size` bytes. El tipo del elemento se obtiene del `TypeRef` interno (`TypeRef::Vector(inner)`) mediante `SemanticType::from_type_ref`, lo que permite determinar el tamaño correcto de cada slot (8 bytes para `Number`, tamaño de puntero para objetos o vectores anidados).

La variante con inicializador, `new T[size](i -> body)`, emite adicionalmente un bucle *cond/body/after*: en cada iteración se liga el índice actual (convertido a `double`) al parámetro de la lambda, se evalúa `body` y el resultado se almacena en la posición correspondiente del buffer. Esto permite inicializar arrays de cualquier tipo, incluyendo arrays de arrays (`new T[][size]`).

6.7.3 Acceso y asignación indexada

El acceso `vec[i]` (`lower_index`) carga el puntero al buffer del `VectorStruct`, convierte el índice `double` a `i64`, verifica en runtime que esté en rango (invocando `hulk_panic` si no), calcula el desplazamiento de bytes `idx * elem_size` y carga el elemento.

La asignación indexada `vec[i] := val (lower_index_assign)` sigue el mismo cálculo de dirección pero emite un `store` en lugar de un `load`. Esto hace posible construir estructuras como matrices 2D: `new Number[] [n]` crea un vector de `n` punteros a `VectorStruct*`, y las filas se asignan individualmente mediante `matrix[i] := new Number[m]`.

6.8 Funciones de Runtime

El compilador delega ciertas operaciones a una biblioteca de runtime enlazada en la fase de linking. Estas funciones se declaran en el módulo LLVM como externas y se resuelven en tiempo de enlazado:

Función	Firma	Propósito
<code>hulk_alloc</code>	<code>(i64) -> i8*</code>	Aloca n bytes en el heap
<code>hulk_concat</code>	<code>(i8*, i8*) -> i8*</code>	Concatena dos cadenas
<code>hulk_panic</code>	<code>(i8*) -> void</code>	Imprime error y termina
<code>hulk_num_to_str</code>	<code>(f64) -> i8*</code>	Convierte número a cadena
<code>hulk_bool_to_str</code>	<code>(i1) -> i8*</code>	Convierte booleano a cadena
<code>hulk_vector_create</code>	<code>(i64) -> ptr</code>	Crea vector con capacidad inicial
<code>hulk_vector_push</code>	<code>(ptr, i8*) -> void</code>	Añade elemento al vector
<code>hulk_vector_get</code>	<code>(ptr, i64) -> i8*</code>	Accede al elemento en índice
<code>printf</code>	<code>libc</code>	Salida estándar (<code>print</code>)

La gestión de memoria del heap usa `malloc` de la `libc` internamente. Los objetos se alocan y nunca se liberan explícitamente (sin GC).

6.9 Funciones

Las funciones globales se procesan en dos pasadas: primero todas las firmas, luego todos los cuerpos. La firma se construye a partir del tipo semántico de cada parámetro y del tipo de retorno registrado en `global_symbols`. Los parámetros se almacenan en `allocas` al inicio del cuerpo.

Gracias a la fase de declaración previa, la recursión directa y mutua funcionan de forma natural: en el momento de compilar el cuerpo de una función, todas las firmas ya están en `self.functions`, por lo que una llamada recursiva simplemente busca la función por nombre y emite un `call` directo a la `FunctionValue` LLVM.

6.10 ABI de Objetos y Vtables

6.10.1 Disposición de memoria

Cada tipo HULK se representa como un `StructType` LLVM heap-alocado:

```
%NombreTipo = type {
    i8*,           ; campo 0: puntero a vtable (posicion fija)
```

```

    <tipo_campo_1>, ; campo 1: primer campo declarado
    <tipo_campo_2>, ; campo 2: segundo campo declarado
    ...
}

```

El campo 0 es invariablemente el puntero a vtable. Esta posición fija es fundamental para el dispatch polimórfico: dado cualquier objeto como `i8*` opaco, siempre es posible castear a `i64**` y obtener el `type_id` sin conocer el tipo concreto.

Para tipos con herencia, los campos del padre se incluyen antes que los propios, garantizando que los índices GEP de los campos heredados sean idénticos en padre e hijo.

6.10.2 Estructura de la vtable y cálculo de índices

La vtable de cada tipo es una variable global LLVM:

```

@vtable.NombreTipo = global {
    i64,    ; slot 0: type_id unico
    i8*,    ; slot 1: puntero al metodo 1
    i8*,    ; slot 2: puntero al metodo 2
    ...
}

```

El orden de los métodos en la vtable sigue un orden canónico estable almacenado en `method_orders`, calculado así:

1. Se toman los métodos del tipo padre en su orden canónico (recursivamente).
2. Se añaden al final los métodos propios del tipo actual que **no** sobrescriben ninguno del padre.
3. Si un método sobrescribe uno del padre, ocupa el **mismo slot** que en el padre; no se añade al final.

Esta propiedad garantiza que el mismo método tiene el mismo índice de slot en padre e hijo, lo que habilita el polimorfismo: el código que invoca `animal.speak()` accede siempre al slot `k` de la vtable, independientemente del tipo concreto de `animal`. El índice final en la vtable es la posición en el orden canónico más 1 (por el slot 0 ocupado por el `type_id`).

6.10.3 Diagrama del ABI

```

Variable (i8*)
|
v
+-----+
| vtable_ptr |-----+

```

campo_1			
campo_2			v
+-----+			+-----+
		type_id (i64)	
		metodo_A (i8*) ----->	fn A()
		metodo_B (i8*) ----->	fn B()
		+-----+	

6.10.4 Generación de llamadas a métodos

Las llamadas a funciones globales se emiten como `call` directo a la `FunctionValue` LLVM. Las llamadas a métodos siguen el dispatch por `vtable`:

1. **Cast tipado:** `bitcast i8*` al struct del tipo.
2. **GEP al campo 0:** obtener la dirección del `vtable_ptr`.
3. **Load:** obtener el `i8*` a la `vtable`.
4. **Cast de vtable:** `bitcast` al tipo de `vtable`.
5. **GEP al slot del método:** índice calculado en compilación.
6. **Load:** obtener el puntero a función.
7. **Cast al tipo de función y call indirecto** con `self` como primer argumento.

El receptor se pasa siempre como primer argumento, convención análoga a la de C++, Python y Rust.

6.11 Herencia

Al construir `new B(args)` donde B hereda de A:

1. Se heap-aloca el struct completo de B.
2. Se almacena `@vtable.B` en el campo 0 (no `@vtable.A`): esto garantiza que aunque la variable sea de tipo A, el dispatch siempre ejecuta las implementaciones de B.
3. Se evalúan y almacenan los argumentos del constructor padre (cláusula `inherits A(...)`) en los campos heredados.
4. Se inicializan los campos propios de B.

Las llamadas a `base.metodo()` emiten un `call` estático directamente a la función del padre, sin pasar por `vtable`, para evitar recursión infinita en métodos sobrescritos.

6.12 Operadores `is` y `as`

El operador `is` compara en runtime el *type_id* del objeto (cargado desde su vtable) con el *type_id* del tipo objetivo, una constante conocida en compilación. El resultado es un `i1`.

El operador `as` realiza la misma comprobación y, si coincide, retorna el puntero casteado al tipo destino mediante `bitcast`. Si no coincide, invoca `hulk_panic` con un mensaje descriptivo. El resultado tiene tipo `ValueKind::Object` con semántico `Custom("T")`.

6.13 Control de Flujo

Condicionales. Las expresiones `if/elif/else` generan una cadena de bloques básicos. Se reserva un `alloca` para el resultado antes de la cadena. Cada condición emite un `br` condicional; todas las ramas almacenan su resultado en el `alloca` compartido y saltan al bloque *merge*, donde se carga el valor final. Esto hace que el `if` sea una expresión con valor independientemente del número de ramas.

While. Se generan tres bloques: *cond* (evalúa la condición y hace `br` condicional), *body* (evalúa el cuerpo y almacena el resultado) y *after* (carga y retorna el resultado). Un `alloca` previo acumula el valor del cuerpo en cada iteración.

For sobre rangos. `range(start, end)` se detecta y genera un contador con `alloca`, cuya condición compara con el límite y cuyo cuerpo lo incrementa en cada iteración.

For sobre vectores. Se accede al campo de longitud del `VectorStruct` para generar el límite del bucle; los elementos se acceden mediante `hulk_vector_get`.

For sobre objetos iterables. Cualquier tipo que implemente `next(): Boolean` y `current(): T` puede usarse como iterable. El bucle emite llamadas por vtable a `next()` en el bloque *cond* y a `current()` en el cuerpo, siguiendo el mismo esquema de tres bloques.

6.14 Clausuras

6.14.1 Representación

Una clausura se representa con un `ClosureStruct` heap-allocado:

```
%ClosureStruct = type {
    i8*,    ; puntero a la funcion thunk
    i8*     ; puntero al entorno capturado
}
```

El entorno capturado es a su vez un struct anónimo heap-alocado que contiene las variables libres de la lambda (variables referenciadas en el cuerpo pero no declaradas como parámetros).

6.14.2 Compilación de una lambda

Para cada lambda se genera una *función thunk* en el módulo LLVM. El thunk recibe el puntero al entorno como primer argumento, seguido de los parámetros explícitos:

```
; Para: (x: Number) => x + capturada
define double @thunk_0(i8* %env_ptr, double %x) {
    %env = bitcast i8* %env_ptr to %env_0*
    %cap = load double, double* getelementptr(%env, 0, 0)
    %r    = fadd double %x, %cap
    ret double %r
}
```

Al construir el `ClosureStruct`:

1. Se heap-aloca un struct con las variables capturadas y se copian sus valores actuales (captura por valor en el momento de creación).
2. Se almacena el puntero al thunk en el campo 0 del `ClosureStruct`.
3. Se almacena el puntero al entorno en el campo 1.

6.14.3 Invocación

Al invocar `closure(args)`:

1. Se cargan `fn_ptr` y `env_ptr` del `ClosureStruct`.
2. Se castea `fn_ptr` al tipo de función del thunk (conocido por el tipo semántico de la clausura en `inferred_types`).
3. Se emite `call fn_ptr(env_ptr, args...)`.

Cuando una función recibe un parámetro de tipo función, el codegen lo trata como `ValueKind::Closure`, permitiendo pasar lambdas a funciones de orden superior de la misma forma que cualquier otro valor.

6.15 Función main y Código de Salida

La función `main` generada tiene siempre la firma `i32 main()`. Todas las expresiones globales del programa se evalúan en orden para sus efectos secundarios y la función retorna siempre `i32 0`. Esta decisión es necesaria porque si se retornara el valor de la última expresión, funciones como `print` retornarían el número de caracteres escritos, produciendo códigos de salida no nulos en programas correctos.

6.16 Manejo de Errores

6.16.1 Errores en tiempo de compilación

Todos los métodos del generador retornan `Result<T, String>`. Los errores se propagan con `?` hasta `codegen_program`, que los devuelve al main. Los principales errores de codegen son tipos no mapeados a `ValueKind`, funciones o métodos no encontrados en las tablas internas (lo que indicaría un fallo previo en la fase de declaración), y errores de construcción de IR por inkwell. Dado que el análisis semántico garantiza la corrección del programa, estos errores indican invariablemente un bug en el propio compilador.

6.16.2 Errores en tiempo de ejecución

Los errores detectables en runtime se manejan invocando `hulk_panic(message)`, que imprime el mensaje por `stderr` y termina el proceso con código 1. Los casos que generan `hulk_panic` son:

- **Cast inválido (as):** el `type_id` del objeto no coincide con el tipo destino.
- **Acceso fuera de rango en vectores:** índice negativo o mayor o igual al tamaño.
- **Dispatch de protocolo sin tipo conforme:** ningún tipo concreto coincide con el `type_id` en un dispatch de protocolo (indica error en la verificación estática).

La división por cero no genera `hulk_panic`: los números son `double` IEEE 754, por lo que la división por cero produce `Inf` o `NaN` de forma silenciosa.

6.17 Optimizaciones y Limitaciones

6.17.1 Optimizaciones

El compilador no implementa optimizaciones propias en el nivel de HULK; delega completamente en el backend de LLVM:

- **mem2reg:** promueve `allocas` a registros SSA; es el paso más impactante, eliminando la mayoría de cargas y almacenamientos redundantes.
- **Inlining:** LLVM puede inlinear funciones pequeñas a nivel `-O2` o superior.
- **SimplifyCFG:** elimina bloques básicos redundantes generados por la traducción sistemática de condicionales y bucles.
- **Eliminación de código muerto:** expresiones sin efectos secundarios cuyo resultado no se usa son candidatas a eliminación.

6.17.2 Limitaciones

- **Sin recolección de basura:** los objetos se alojan con `hulk_alloc` pero nunca se liberan; un programa que cree muchos objetos consumirá memoria indefinidamente.
- **Módulo único:** todo el programa se compila en un único módulo LLVM, impidiendo la compilación incremental.
- **Sin optimización de llamadas en cola:** la recursión profunda puede desbordar la pila.
- **Dispatch lineal en protocolos:** la selección del tipo concreto en un dispatch de protocolo escala linealmente con el número de tipos conformes; en programas con muchos tipos esto podría suponer un coste no despreciable frente a una tabla de interfaces dedicada.
- **Imports por recompilación:** los módulos importados se recompilan completos en cada invocación, sin caché de artefactos intermedios.

7 Características Adicionales del Lenguaje

Esta sección describe las características del lenguaje HULK que van más allá del núcleo obligatorio. Para cada una se analiza cómo fue extendido cada nivel del compilador, las decisiones de diseño adoptadas y su relación con construcciones equivalentes en otros lenguajes de programación.

7.1 Lambdas y Clausuras

7.1.1 Motivación

Las funciones de primera clase permiten pasar comportamiento como dato, habilitando patrones de programación funcional. La decisión fue tratar las lambdas como *clausuras*: funciones que pueden capturar variables de su entorno léxico.

7.1.2 Extensión del Compilador

Lexer y AST. No se requirieron nuevos tokens; el operador `=>` ya existía para funciones de una expresión. En el AST se agregó `LambdaExpr` con lista de parámetros tipados y expresión de cuerpo. Las lambdas son expresiones: pueden aparecer en cualquier posición donde se espere un valor.

Parser. Se añadió la regla:

```
LambdaExpr ::= "(" Params ")" ">" Expr  
            | Ident ">" Expr
```

La segunda forma es azúcar para lambdas de un solo parámetro.

Análisis Semántico. El analizador infiere el tipo de una lambda como `SemanticType::Function(param_types, ret_type)`. La captura de variables del entorno se registra inspeccionando las variables libres del cuerpo respecto al scope en que aparece la lambda. Las lambdas son compatibles con cualquier tipo de función de la misma aridad y tipos, lo que permite pasarlas como argumentos directamente.

Generación de Código. Las clausuras se representan como un `ClosureStruct`:

```
%ClosureStruct = type { i8*, i8* }
                  ;      ^fn_ptr ^env_ptr
```

Para cada lambda se emite una *función thunk* en el módulo LLVM que recibe el puntero al entorno como primer argumento. El entorno capturado se heap-aloca y se almacena como un struct anónimo con las variables capturadas. La invocación de una clausura carga el puntero a función y el entorno desde el `ClosureStruct` y emite un `call` indirecto.

7.1.3 Comparativa con Otros Lenguajes

- **Haskell:** las funciones son ciudadanas de primera clase por diseño; el currying es automático. HULK no soporta currying pero sí captura léxica similar al lambda calculus en que se basa Haskell.
- **Python:** lambda está restringido a una sola expresión. HULK permite lambdas con cuerpos arbitrarios, más cercano a las *arrow functions* de **JavaScript/TypeScript**, que también capturan el `this`/scope léxico.
- **Java:** hasta Java 7 no existían lambdas; desde Java 8 se implementan como interfaces funcionales (`@FunctionalInterface`), es decir, objetos en *disguise*. HULK las representa como structs explícitos, similar a cómo el compilador de Java genera clases anónimas internas.
- **Rust:** los cierres usan los traits `Fn`, `FnMut`, `FnOnce` según cómo capturan. HULK simplifica esto usando siempre captura por referencia (puntero al entorno heap-alocado), sacrificando eficiencia a cambio de simplicidad de implementación.
- **C:** no tiene clausuras nativas. Las funciones de orden superior se simulan con punteros a función, sin captura de entorno, lo que obliga a pasar el estado manualmente. El modelo de HULK es estrictamente más expresivo.

7.2 Protocolos (Interfaces Estructurales)

7.2.1 Motivación

Los protocolos permiten escribir código polimórfico sin necesidad de herencia. La decisión de diseño clave fue adoptar **tipado estructural implícito**: un

tipo conforma a un protocolo si y solo si implementa todos sus métodos, sin declaración explícita de conformancia.

7.2.2 Extensión del Compilador

Lexer y AST. Se añadió la palabra reservada `protocol`. El nodo `ProtocolDecl` contiene únicamente firmas de métodos (nombre, parámetros y tipo de retorno), sin cuerpos.

Parser. La regla distingue `protocol` de `type`: los protocolos no tienen campos ni inicializadores, solo firmas:

```
ProtocolDecl ::= "protocol" Ident "{" MethodSig* "}"
MethodSig    ::= Ident "(" Params ")" ":" TypeRef ";"
```

Análisis Semántico. Los protocolos se almacenan en `protocol_shapes`, separados de `type_shapes`. La conformancia se verifica en el sitio de uso: cuando una expresión de tipo `T` se asigna a una variable de tipo protocolo `P`, el analizador comprueba que `T` tenga todos los métodos de `P` con tipos compatibles. No hay anotación de conformancia en la declaración del tipo, de modo que un tipo puede conformar a múltiples protocolos sin modificación.

Generación de Código. Los protocolos son una construcción exclusivamente estática: no tienen representación en tiempo de ejecución. Una variable de tipo protocolo contiene en realidad un puntero a un objeto concreto. El dispatch se resuelve mediante un *switch* sobre el *type_id* del objeto receptor, comparándolo contra los *type_ids* de todos los tipos concretos que conforman el protocolo (calculados en tiempo de compilación). Si ninguno coincide se invoca `hulk_panic`. Este enfoque elimina la necesidad de una tabla de interfaces en runtime.

7.2.3 Comparativa con Otros Lenguajes

- **Go:** los interfaces de Go son el referente más directo. También son estructurales e implícitos. En runtime, Go usa una *itable* (tabla de punteros a métodos por combinación tipo/interface). HULK prescinde de esta tabla usando el switch de *type_id*, lo que es más simple pero escala peor con muchos tipos conformes.
- **TypeScript:** también usa tipado estructural para interfaces. La verificación es puramente estática (el runtime es JavaScript). HULK sigue el mismo principio de “las interfaces se borran en compilación”.
- **Java / C#:** usan tipado nominal: un tipo debe declarar explícitamente que implementa una interfaz (`implements/:`). Esto requiere modificar la declaración del tipo, lo que impide extender tipos de terceros. El tipado estructural de HULK es más flexible en este sentido.

- **Haskell:** las *typeclasses* son el mecanismo equivalente. La conformancia sí debe declararse explícitamente (**instance**), pero el mecanismo de dispatch usa *dictionaries* pasados implícitamente, similar conceptualmente al switch de *type_id* de HULK.
- **Rust:** los traits son nominales (se declara `impl Trait for Type`). El dispatch puede ser estático (monomorphization) o dinámico (`dyn Trait`, que usa vtable). HULK combina verificación estructural (como Go) con dispatch dinámico (como `dyn Trait`).

7.3 Macros

7.3.1 Motivación

Los macros permiten abstracciones sintácticas que no son expresables con funciones ordinarias: transforman fragmentos de AST en tiempo de compilación, antes del análisis de tipos. La decisión fue implementar macros **higiénicas** por sustitución de AST, evitando los problemas de captura de nombres del preprocesador de C.

7.3.2 Extensión del Compilador

Lexer y AST. Se añadieron los tokens `define` y `!` (marcador de invocación). El AST incluye `MacroDecl` (nombre, parámetros formales y plantilla de cuerpo) y `MacroCallExpr` (nombre e argumentos como sub-ASTs sin tipar).

Lexer — Transformaciones de Tokens. Un aspecto distintivo es que parte del reconocimiento de macros ocurre a nivel de token: se aplican varias pasadas de transformación sobre la secuencia de tokens antes de pasarla al parser (`mark_macro_calls`, `add_lambda_block_tokens`, etc.). Esto simplifica la gramática al elevar a nivel léxico ciertas ambigüedades contextuales.

Parser. Los macros se parsean como ítems de primer nivel con la misma precedencia que `type` y `function`:

```
MacroDecl ::= "define" Ident "(" Idents ")" Expr
```

Las invocaciones se distinguen en el parser por el sufijo `!`.

Expansión — Fase Propia. Se implementó una fase de expansión de macros entre el parsing y el análisis semántico (`macro_expander`). Esta fase recorre el AST, detecta invocaciones de macros y las sustituye por el cuerpo del macro con los argumentos reemplazados. La higiene se implementa renombrando las variables internas del macro con identificadores frescos en cada expansión, evitando captura accidental.

Análisis Semántico y Codegen. Tras la expansión, el AST no contiene nodos de macro: el analizador semántico y el generador de código operan sobre el AST expandido ordinario, sin conocimiento de que hubo macros. Esto simplifica enormemente ambas fases.

7.3.3 Comparativa con Otros Lenguajes

- **C/C++:** el preprocesador (`#define`) opera sobre texto puro, antes del lexer. No es higiénico: la captura de nombres es un error clásico. HULK opera sobre AST, eliminando este problema.
- **Lisp/Scheme:** los macros son la característica definitoria del lenguaje. En Scheme, los `syntax-rules` son higiénicos por diseño. El modelo de HULK es comparable: sustitución de AST con renombramiento, aunque sin la potencia de los macros Lisp que permiten computación arbitraria en tiempo de expansión.
- **Rust:** los macros declarativos (`macro_rules!`) son higiénicos y operan sobre tokens, no texto. Los macros procedurales operan sobre `TokenStream` y permiten computación completa. HULK es más simple que Rust pero más seguro que C.
- **Julia:** los macros de Julia operan sobre expresiones del lenguaje (también representadas como AST), similar a HULK, y son parte integral del ecosistema para metaprogramación numérica.
- **Java/Python:** no tienen macros. Las abstracciones sintácticas deben hacerse con funciones de orden superior o generación de código externa, lo que es menos expresivo.

7.4 Generadores e Iterables Personalizados

7.4.1 Motivación

El bucle `for` del núcleo opera sobre rangos numéricos y vectores. Los generadores extienden esto a cualquier objeto que implemente el protocolo `Iterable`: métodos `next(): Boolean` y `current(): T`. Esto permite definir secuencias potencialmente infinitas de forma perezosa, sin materializar todos los elementos en memoria.

7.4.2 Extensión del Compilador

Lexer y AST. No se requirieron cambios: la sintaxis `for (x in expr)` ya existía. La semántica del iterable es una propiedad del tipo de `expr`, no de la sintaxis.

Análisis Semántico. Se añadió la verificación de que el tipo de la expresión iterable, si no es un vector ni `range`, implemente el protocolo `Iterable` (presencia de `next` y `current` con las firmas correctas). El tipo de la variable de iteración se infiere del tipo de retorno de `current()`.

Generación de Código. En codegen se discriminan tres rutas según el tipo inferido del iterable: `range`, vector, u objeto/protocolo. Para objetos concretos, el bucle emite llamadas por vtable a `next()` y `current()` en cada iteración. Para iterables de tipo protocolo, se usa `emit_protocol_dispatch`. La estructura de bloques IR (*cond*, *body*, *after*) es idéntica en ambos casos.

Evaluación Perezosa. Un generador no calcula sus elementos por adelantado: `next()` y `current()` se invocan una vez por iteración. El estado del generador se mantiene en los campos del objeto (el heap-alloc del constructor). Esto permite, por ejemplo, un generador de números primos que opera indefinidamente sin agotar la memoria.

7.4.3 Comparativa con Otros Lenguajes

- **Python:** los generadores usan `yield`, que suspende la ejecución de una función y la reanuda en la siguiente iteración. Este modelo requiere soporte de continuaciones o coroutines en el runtime. HULK evita esta complejidad usando el patrón iterator con estado explícito.
- **C# / .NET:** `IEnumerable<T>` e `IEnumerator<T>` son exactamente el equivalente nominal del protocolo `Iterable` de HULK. C# también soporta `yield return` para generadores sin estado explícito. HULK adopta solo el patrón de interfaz, no el `yield`.
- **Java:** `Iterable<T>` e `Iterator<T>` son los interfaces análogos. El bucle `for-each` de Java desazucara exactamente a llamadas `hasNext()/next()`, el mismo patrón que `next()/current()` de HULK.
- **Rust:** el trait `Iterator` define un único método `next() -> Option<T>`, combinando la detección de fin y la obtención del elemento en una sola llamada. HULK los separa en `next()` y `current()`, lo que duplica las llamadas por iteración pero hace la implementación de generadores más intuitiva.
- **Haskell:** las listas son perezosas por defecto; cualquier lista es conceptualmente un generador infinito. HULK no tiene evaluación perezosa nativa pero aproxima el comportamiento mediante iteradores con estado explícito.

7.5 Sistema de Módulos e Importaciones

7.5.1 Motivación

Sin un sistema de módulos, todo el código debe residir en un único archivo. Las importaciones permiten dividir el programa en unidades compilables independientes, reutilizar tipos y funciones entre archivos y organizar proyectos de mayor tamaño. La decisión de diseño fue implementar un sistema de **importación por recompilación**: cada módulo importado se vuelve a parsear y analizar en el contexto del compilador, sin caché ni artefactos intermedios.

7.5.2 Extensión del Compilador

Lexer y AST. Se añadió la palabra reservada `import`. El nodo `ImportDecl` contiene únicamente el nombre del módulo como cadena. Los imports se tratan como ítems de primer nivel, al mismo nivel que `type`, `function` y `protocol`.

Parser. La gramática es deliberadamente simple:

```
ImportDecl ::= "import" StringLiteral ";"
```

El nombre del módulo es una ruta relativa al archivo fuente. Esta simplicidad evita la necesidad de un sistema de resolución de paquetes durante el parsing.

Análisis Semántico. Al analizar un programa, el analizador detecta los `ImportDecl` y carga cada módulo referenciado: lee el archivo, lo lexea, lo parsea y le aplica su propio análisis semántico de forma recursiva. Los símbolos resultantes (funciones, tipos, protocolos) se fusionan en el namespace del módulo importador. Se mantiene un conjunto de módulos ya visitados para evitar ciclos de importación.

Generación de Código. La fase de codegen implementa la función `collect_imported_module_units`, que recorre los `ImportDecl` del programa recursivamente y retorna una lista de `ImportedModuleUnit`: structs que contienen el AST y el resultado del análisis semántico de cada módulo importado. Antes de compilar el módulo principal, el generador:

1. Ejecuta `collect_type_decls` y `prepare_object_types` para los tipos de cada módulo importado, garantizando que los structs LLVM estén disponibles antes de compilar el módulo principal.
2. Ejecuta `declare_functions` para cada módulo importado, de modo que las funciones importadas sean referenciables por el módulo principal como llamadas directas.

Todos los cuerpos se emiten en un único módulo IR, simplificando el enlazado a costa de tiempos de compilación mayores en proyectos grandes.

7.5.3 Comparativa con Otros Lenguajes

- **C/C++:** el sistema de `#include` es textual: el preprocesador copia el contenido del archivo incluido en el punto de inclusión. Esto produce los problemas clásicos de inclusión múltiple (resueltos con *include guards*) y tiempos de compilación elevados. HULK evita esto al operar sobre AST y mantener un registro de módulos ya procesados.
- **Java:** el sistema de paquetes es nominal (`import com.ejemplo.Clase`); el compilador resuelve la ruta en el classpath. La unidad de compilación es el archivo `.java`, y los artefactos intermedios (`.class`) se cachean. HULK usa rutas relativas simples y no produce artefactos intermedios.
- **Python:** `import modulo` busca el módulo en `sys.path` y lo ejecuta completamente la primera vez, cacheando el resultado en `sys.modules`. El modelo de HULK es similar en que la importación dispara el procesamiento completo del módulo, pero sin caché entre ejecuciones del compilador.
- **Rust:** el sistema de módulos es jerárquico (`mod`, `use`) y el compilador gestiona dependencias entre crates con resolución incremental. Es el sistema más sofisticado de los comparados, con caché de artefactos intermedios. HULK no implementa caché entre compilaciones, lo que simplifica la implementación pero aumenta los tiempos de compilación en proyectos grandes.